

RegRadar Technical Architecture Document

Contents

RegRadar — Technical Architecture Document	1
1. Recommended Tech Stack (with Reasoning)	1
2. Complete File & Folder Structure	5
3. Full Database Schema	8
4. Environment Variables & Configuration Notes	14
5. What This Document Deliberately Doesn't Cover	16

RegRadar — Technical Architecture Document

Version 1.0 | Prepared for the RegRadar V1 build (8-12 week timeline)

This document translates the RegRadar Project Brief and PRD into an implementation-ready architecture: the tech stack and why each piece was chosen, the full repository layout, the complete database schema, and the environment/configuration checklist you need before writing the first line of code.

1. Recommended Tech Stack (with Reasoning)

The PRD already names most of these tools. What follows is the reasoning a senior architect would want written down before committing — including a few additions the PRD doesn't spell out (auth, migrations, testing, CI/CD) that you'll need in practice.

1.1 Orchestration & Agents

Choice	Reasoning
LangGraph 0.2	You need a supervisor coordinating 6 specialized agents with conditional branching (e.g., skip RAG for low-priority filings, retry a failed extraction, pause for human review on Critical filings). Vanilla LangChain chains are linear and don't give you typed shared state or per-node retry — LangGraph's graph model does. The steeper learning curve is worth it because the whole product's value proposition is agent orchestration; this is not a place to cut a corner.

Choice	Reasoning
Typed shared state (Pydantic models as the LangGraph state schema)	Each agent reads/writes a subset of one typed object instead of passing ad-hoc dicts. This is what makes agents “independently swappable” — you can replace the summarization model without any other agent’s code changing, because the contract is the state schema, not the implementation.

1.2 LLMs & Classification

Choice	Reasoning
GPT-4o (structured output / JSON mode) for the Analysis Agent	Structured extraction (obligations, deadlines, risk flags as a typed schema) needs strict JSON-schema adherence. GPT-4o’s native structured output mode is currently the most reliable option for this without hand-rolled output parsers and retry-on-malformed-JSON logic.
HF facebook/bart-large-mnli for zero-shot domain classification	No labeled training data exists on day one. Zero-shot lets you ship triage in week 3 instead of waiting on a labeling effort. Revisit with a fine-tuned classifier once you’ve accumulated human-reviewed labels (ADR-02 in the PRD already flags this as a V2 upgrade path).
HF facebook/bart-large-cnn + Granite-13B fallback for summarization	Summarization is high-volume and cost-sensitive (every filing gets one), so it’s the right place to use a cheaper open model instead of GPT-4o. Granite acts as the fallback tier for Low/Medium priority filings — this is what makes the tiered-routing cost model in ADR-04 work.
OpenAI text-embedding-3-small	1536 dimensions, fast, and inexpensive at the volume implied by 500K+ indexed filings. There’s no retrieval-quality reason to reach for a larger embedding model at this corpus size.

1.3 RAG Layer

Choice	Reasoning
LlamaIndex 0.10	Best chunking primitives in the ecosystem plus built-in hybrid (BM25 + dense) search and re-ranking, which the PRD explicitly requires for regulatory PDFs that mix prose, tables, and footnotes.

Choice	Reasoning
pgvector (Postgres extension) over Pinecone/Weaviate Chunking: 512-token chunks, 50-token overlap, section-boundary aware	Filing metadata, obligations, and vectors are all naturally relational to each other — a filing has one set of chunks, and every chunk belongs to exactly one filing. Keeping vectors in the same database as the relational data avoids a second system to operate, back up, and keep in sync. The tradeoff (pgvector is roughly 2x slower than a dedicated vector DB past ~10M vectors) is acceptable at the stated corpus size; if the corpus grows past that, this is the first component to revisit. Regulatory filings have legally meaningful section boundaries (e.g., “Item 1A. Risk Factors”). Chunking blindly by token count risks splitting a sentence that a compliance reader needs intact. Overlap protects against losing context at chunk edges during retrieval.

1.4 Data Ingestion

Choice	Reasoning
Prefect 2.x SEC EDGAR API, FDA RSS, FINRA feeds ETag / Last-Modified based polling	You need scheduled polling (every 5 minutes) with built-in retry, backoff, and run observability — writing this yourself with <code>cron</code> + custom retry logic is reinventing what Prefect already does well, and Prefect’s UI gives you ingestion-pipeline visibility for free. These are the canonical, free, machine-readable sources for each regulator. EDGAR has a documented 10 req/sec rate limit (PRD risk R2) — Prefect’s retry/backoff handles this without custom rate-limit code. Avoids re-downloading unchanged content and gives you a cheap way to detect “is this actually new” without diffing full documents.

1.5 Serving & Async Processing

Choice	Reasoning
FastAPI + Pydantic v2 Redis + Celery	Async-native, typed request/response models, and automatic OpenAPI docs — all three matter here because the product is explicitly API-first (NG3: no UI in V1) and needs a clean versioned contract for the Engineering Lead persona. Decouples ingestion (which happens on a schedule) from the agent pipeline (which is triggered per-filing and takes variable time depending on document length and model tier). Without a queue, a slow GPT-4o call on one filing would block ingestion of the next.

Choice	Reasoning
Rate limiting at the API layer (e.g., <code>slowapi</code> or a Redis-backed token bucket)	Not explicit in the PRD but required by G7 (“versioned REST API with <500ms P99 latency”) — without rate limiting, one noisy API consumer can degrade P99 for everyone.

1.6 Storage

Choice	Reasoning
Supabase (managed Postgres + pgvector)	Gives you managed Postgres, the pgvector extension, connection pooling, and daily backups without standing up your own RDS/pgvector install. Also gives a free row-level-security layer if you add multi-tenant auth later.
S3 for raw PDFs	PRD is explicit (NG2): RegRadar is not a document management system. Raw PDFs are large, immutable blobs that don’t belong in a relational DB — S3 (or an S3-compatible store) is the standard choice, and Postgres just stores the S3 key/URL.

1.7 Evaluation & Observability

Choice	Reasoning
Ragas (faithfulness, context recall)	Purpose-built RAG eval metrics — faithfulness catches hallucinated claims in briefs, context recall catches retrieval misses. Both map directly to PRD success metrics (R1 mitigation, G8).
HF evaluate (ROUGE-L)	Standard, well-understood summarization quality metric against human-written reference briefs.
LangSmith	Prompt versioning and regression testing — “every prompt change runs a regression suite before it ships” (PRD §10) requires a system that can diff prompt versions and gate deploys on eval scores.
Langfuse	Cost-per-filing and latency tracking, plus the \$50/day budget-alert circuit breaker (PRD risk R4). Langfuse and LangSmith overlap somewhat; running both is intentional redundancy — LangSmith for prompt/eval workflow, Langfuse for cost/latency dashboards. If budget is tight, Langfuse alone can cover both with more manual eval wiring.

1.8 Delivery

Choice	Reasoning
Slack Webhooks	Simplest reliable path to the CCO/Analyst personas' primary touchpoint; no Slack app approval process needed for incoming webhooks.
SendGrid	Handles the weekly digest and email alert paths with delivery tracking, which you'd otherwise have to build on raw SMTP.
HMAC-signed outbound webhooks	Required by US-05 / the API contract (POST /v1/webhooks). Signing payloads lets Engineering Lead consumers verify authenticity — standard practice for any product that fires webhooks to third-party endpoints.

1.9 Deployment & Infra

Choice	Reasoning
Docker	Every component (API, Celery workers, Prefect flows) needs an identical runtime across local dev, CI, and production — non-negotiable for a multi-process system like this.
Fly.io	Cheap always-on compute for both the API and background workers, with simple multi-region deploy if latency to EDGAR/FDA ever matters. Railway is a reasonable substitute if you prefer its dashboard/DX; the architecture doesn't depend on which one you pick.
GitHub Actions (CI/CD) — <i>not named in the PRD, added here</i>	You need somewhere to run the eval regression suite (§1.7) on every prompt/PR change and block merges on failing evals ("a failing eval blocks the deploy" — PRD §10). GitHub Actions is the natural default if the repo lives on GitHub.
Alembic (DB migrations) — <i>not named in the PRD, added here</i>	Any schema with 7+ interrelated tables needs versioned migrations from day one, not CREATE TABLE scripts run by hand. Alembic is the standard for SQLAlchemy-based Python projects and integrates cleanly with Supabase Postgres.

1.10 Auth (gap in the PRD — recommended for V1, not just V2)

The PRD marks SSO/enterprise auth as a V2 cut item, which is correct — but a versioned public API with webhook registration and per-consumer rate limiting still needs *some* authentication in V1, otherwise anyone can register a webhook or hit /v1/filings. Recommend a simple **API key** model (a table of hashed keys, checked via a FastAPI dependency) scoped per Engineering Lead consumer. This is intentionally lightweight — full SSO/RBAC can wait for multi-tenant V2, but bare API-key auth is a half-day of work and closes an obvious hole.

2. Complete File & Folder Structure

regradar/

```

└─ .github/
  └─ workflows/
    └─ ci.yml # lint, unit tests, type-check on every PR
    └─ eval-regression.yml # runs Ragas/ROUGE suite, blocks merge on failure
    └─ deploy.yml # build image, push, deploy to Fly.io on main

└─ src/
  └─ regradar/
    └─ __init__.py

    └─ agents/ # LangGraph agent implementations
      └─ __init__.py
      └─ graph.py # supervisor graph: edges, conditional routing
      └─ state.py # typed shared state (Pydantic) between agents
      └─ ingestion_agent.py
      └─ triage_agent.py
      └─ rag_retrieval_agent.py
      └─ analysis_agent.py
      └─ summarization_agent.py
      └─ delivery_agent.py

    └─ ingestion/ # Prefect flows + source connectors
      └─ __init__.py
      └─ flows.py # Prefect flow definitions (poll every 5 min)
      └─ sources/
        └─ sec_edgar.py
        └─ fda_rss.py
        └─ finra_feed.py
      └─ pdf_intake.py # download, hash, dedupe, push to S3

    └─ rag/
      └─ __init__.py
      └─ chunking.py # 512-token / 50-overlap, section-boundary aware
      └─ embeddings.py # text-embedding-3-small wrapper
      └─ retriever.py # LlamaIndex hybrid BM25 + dense retriever
      └─ reranker.py

    └─ models/ # SQLAlchemy ORM models (mirrors §3 schema)
      └─ __init__.py
      └─ filing.py
      └─ chunk.py
      └─ extraction.py
      └─ brief.py
      └─ webhook.py
      └─ delivery.py
      └─ eval_run.py
      └─ source_config.py
      └─ api_key.py

    └─ schemas/ # Pydantic request/response models (API contract)
      └─ __init__.py
      └─ filing.py
      └─ webhook.py
      └─ metrics.py
      └─ config.py

    └─ api/ # FastAPI application

```

```

├── __init__.py
├── main.py                # app factory, middleware, startup/shutdown hooks
├── deps.py                # DB session, API-key auth, rate limiter
├── routers/
│   ├── filings.py        # GET /v1/filings, /filings/{id}, /search, /brief
│   ├── webhooks.py       # POST/DELETE /v1/webhooks
│   ├── metrics.py        # GET /v1/metrics
│   └── config.py         # POST /v1/config/sources
├── middleware/
│   ├── rate_limit.py
│   └── request_logging.py
├── workers/               # Celery tasks
│   ├── __init__.py
│   ├── celery_app.py
│   ├── pipeline_tasks.py # runs the LangGraph pipeline per filing
│   └── digest_tasks.py   # weekly digest job (US-07)
├── delivery/
│   ├── __init__.py
│   ├── slack_client.py
│   ├── sendgrid_client.py
│   └── webhook_dispatcher.py # HMAC signing + POST with retry
├── eval/
│   ├── __init__.py
│   ├── ragas_suite.py    # faithfulness, context recall
│   ├── rouge_suite.py
│   ├── alert_precision_recall.py
│   ├── extraction_fl.py
│   └── fixtures/         # labeled test sets (100 baseline, 200 alert set)
├── llm_routing/
│   ├── __init__.py
│   └── tiered_router.py   # Critical/High -> GPT-4o, Low/Medium -> Granite
├── observability/
│   ├── __init__.py
│   ├── langsmith_config.py
│   └── langfuse_config.py
├── core/
│   ├── __init__.py
│   ├── config.py         # Pydantic Settings – loads all env vars (see §4)
│   ├── db.py             # SQLAlchemy engine/session factory
│   ├── redis_client.py
│   └── s3_client.py
├── cli.py                 # local dev entrypoints (run one filing manually, etc.)
├── migrations/           # Alembic
│   ├── env.py
│   └── versions/
├── tests/
│   ├── unit/
│   └── agents/

```

```

├── rag/
│   └── delivery/
├── integration/
│   ├── test_api_endpoints.py
│   ├── test_pipeline_end_to_end.py
│   └── test_webhook_hmac.py
├── fixtures/
│   └── sample_filings/          # edge-case PDFs: scanned, tabular, footnotes (R3)
├── docs/
│   ├── adrs/
│   │   ├── ADR-01-pgvector-over-pinecone.md
│   │   ├── ADR-02-zero-shot-over-fine-tuned.md
│   │   ├── ADR-03-langgraph-over-langchain.md
│   │   └── ADR-04-tiered-model-routing.md
│   ├── system-diagram.png
│   └── api-contract.md
├── infra/
│   ├── docker/
│   │   ├── Dockerfile.api
│   │   ├── Dockerfile.worker
│   │   └── Dockerfile.prefect
│   ├── docker-compose.yml      # local dev: postgres+pgvector, redis, api, worker
│   └── fly/
│       ├── fly.api.toml
│       └── fly.worker.toml
├── .env.example                # see §4 – template, no real secrets committed
├── pyproject.toml
├── alembic.ini
├── README.md
└── CHANGELOG.md

```

A few structural notes worth calling out:

The `agents/` and `workers/` split matters: `agents` define *what* each step does and are unit-testable in isolation with mocked state; `workers/pipeline_tasks.py` is the thin Celery wrapper that actually invokes the LangGraph graph asynchronously. Keeping the LangGraph definition free of Celery-specific code means you can also run the whole pipeline synchronously from `cli.py` for local debugging, without a Redis broker running.

`models/` (SQLAlchemy) and `schemas/` (Pydantic) are deliberately separate. The ORM models are the source of truth for what's in the database; the Pydantic schemas are the API's public contract. Collapsing these into one set of classes is a common shortcut that becomes painful the first time you want to change a DB column without also changing the API response shape.

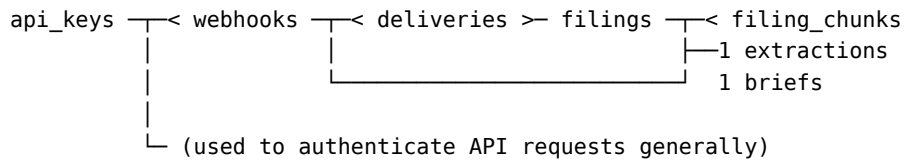
3. Full Database Schema

All tables live in the Supabase Postgres instance with the `pgvector` extension enabled. Below is each table, its fields, and — in plain English — what it's for and how it relates to the others.

Entity-relationship summary (plain English)

A **filing** is the central record — one row per regulatory document ingested. Each filing is split into many **filing_chunks** for retrieval (one filing → many chunks). Each filing has exactly one **extraction** record

(the structured obligations/deadlines/risk data pulled out by the Analysis Agent) and exactly one **brief** record (the persona-tailored summaries). A filing can trigger many **deliveries** (one Slack message, one email, and one webhook POST per registered webhook — so one filing can have several delivery rows, each tied optionally to a **webhook**). **webhooks** are registered independently by API consumers and aren't tied to any single filing. **source_configs** control which regulators/domains the ingestion pipeline watches — independent of any filing. **eval_runs** are periodic snapshots of system quality metrics, not tied to a specific filing. **api_keys** authenticate API consumers and are referenced by webhooks (who registered it) and can optionally be logged against deliveries/API requests for auditing.



source_configs (standalone – controls ingestion)

eval_runs (standalone – periodic quality snapshots)

3.1 filings

The core table — one row per regulatory document.

Field	Type	Notes
id	UUID, PK	
source	ENUM(SEC, FDA, FINRA)	Which regulator this came from
source_document_id	TEXT	The regulator's own ID/accession number, used for dedup
entity_name	TEXT	The company/entity the filing concerns
filing_type	TEXT	e.g., "10-K", "8-K", "FDA Guidance", "FINRA Notice"
filing_url	TEXT	Original source URL
raw_pdf_s3_key	TEXT	Pointer to the immutable PDF in S3 (PDFs themselves are never stored in Postgres — NG2)
published_at	TIMESTAMPTZ	When the regulator published it
ingested_at	TIMESTAMPTZ	When RegRadar's Ingestion Agent picked it up
status	ENUM(ingested, classifying, retrieving, analyzing, summarizing, delivering, complete, failed)	Pipeline state — lets the API and any future dashboard show where a filing is in the DAG
domain	ENUM(financial, clinical, environmental, other)	Set by the Triage Agent (zero-shot classification)
risk_level	ENUM(low, medium, high, critical)	Set by the Triage Agent, may be revised by the Analysis Agent
priority_score	FLOAT	Raw confidence/priority score behind risk_level, kept for eval and dual-model-voting audits (R5)

Field	Type	Notes
classification_confidence	FLOAT	Zero-shot model's confidence, used to decide if GPT-4o spot-check voting is triggered (R5)
processing_error	TEXT, nullable	Last error message if status = failed
created_at / updated_at	TIMESTAMPTZ	Standard audit columns

In plain English: this is the “one row per document” table everything else hangs off. status exists so you can always answer “where is this filing in the pipeline right now” without inferring it from which downstream tables are populated.

3.2 filing_chunks

Many rows per filing — the RAG index.

Field	Type	Notes
id	UUID, PK	
filing_id	UUID, FK → filings.id	
chunk_index	INTEGER	Order within the document
chunk_text	TEXT	The raw chunk content
section_reference	TEXT, nullable	e.g., “Item 1A. Risk Factors” — preserved because chunking is section-boundary aware
token_count	INTEGER	For debugging
embedding	VECTOR(1536)	chunking-strategy changes pgvector column, text-embedding-3-small output
is_table	BOOLEAN	Flags chunks that came from a table/exhibit, so Table QA logic can treat them differently
created_at	TIMESTAMPTZ	

In plain English: every filing is broken into overlapping 512-token pieces, each turned into a vector, so the RAG Retrieval Agent can find “similar past filings” by comparing vectors instead of re-reading every document from scratch.

3.3 extractions

One row per filing — the Analysis Agent’s structured output.

Field	Type	Notes
id	UUID, PK	
filing_id	UUID, FK → filings.id, UNIQUE	One-to-one with filings
obligations	JSONB	List of extracted obligations, each with a source citation (required by R1 mitigation)
deadlines	JSONB	List of {description, date} objects

Field	Type	Notes
risk_flags	JSONB	List of flagged risk items
affected_products	JSONB	List of product names/lines affected
key_entities	JSONB	Named entities pulled from the filing
competitor_mentions	JSONB	List of competitor names mentioned (US-08 support)
model_used	TEXT	Which model produced this (GPT-4o vs. fallback) — for cost/quality auditing
raw_model_response	JSONB	Full raw structured output, kept for eval/debugging without re-calling the LLM
created_at	TIMESTAMPTZ	

In plain English: this is the “what does this filing actually require us to do” table — everything an Analyst needs to avoid reading all 80 pages (US-02).

3.4 briefs

One row per filing — the Summarization Agent’s output.

Field	Type	Notes
id	UUID, PK	
filing_id	UUID, FK → filings.id, UNIQUE	One-to-one with filings
executive_brief	TEXT	The 3-5 sentence plain-English brief (G3)
cco_summary	TEXT	Persona-tailored for Chief Compliance Officer
analyst_summary	TEXT	Persona-tailored for Compliance Analyst
engineer_summary	TEXT	Persona-tailored for Engineering Lead
model_used	TEXT	bart-large-cnn vs. Granite vs. GPT-4o fallback
rouge_l_score	FLOAT, nullable	Populated by the eval harness against reference briefs, when available
created_at	TIMESTAMPTZ	

In plain English: the four different summaries exist because each persona (§4 of the PRD) wants a different depth and framing of the same filing — the CCO wants three sentences, the Engineering Lead cares that a webhook fired.

3.5 webhooks

Registered by API consumers (Engineering Lead persona), independent of any single filing.

Field	Type	Notes
id	UUID, PK	

Field	Type	Notes
api_key_id	UUID, FK → api_keys.id	Who registered this webhook
url	TEXT	Destination endpoint
hmac_secret	TEXT	Used to sign outbound payloads so the consumer can verify authenticity
is_active	BOOLEAN	Soft-disable without deleting (preserves delivery history)
filter_domain	TEXT, nullable	Optional filter — only fire for a specific domain
filter_min_risk	ENUM(low,medium,high,critical), nullable	Optional filter — only fire above a risk threshold
created_at	TIMESTAMPTZ	

In plain English: this is the address book for POST /v1/webhooks — every row is one external system that wants a POST every time a new filing finishes processing (US-05).

3.6 deliveries

Many rows per filing — one per actual delivery attempt across all channels.

Field	Type	Notes
id	UUID, PK	
filing_id	UUID, FK → filings.id	
channel	ENUM(slack, email, webhook)	
webhook_id	UUID, FK → webhooks.id, nullable	Populated only when channel = webhook
recipient	TEXT	Slack channel ID, email address, or webhook URL snapshot
status	ENUM(pending, sent, failed, retrying)	
response_code	INTEGER, nullable	HTTP status for webhook/email delivery
attempt_count	INTEGER, default 0	
sent_at	TIMESTAMPTZ, nullable	
created_at	TIMESTAMPTZ	

In plain English: this is the audit trail the Delivery Agent writes to — “did the Critical alert for filing X actually reach Slack, and when.” Also what powers the weekly digest job (US-07), which queries all deliveries in the trailing week.

3.7 source_configs

Standalone table — controls what the Ingestion Agent watches (US-10, POST /v1/config/sources).

Field	Type	Notes
id	UUID, PK	
source	ENUM(SEC, FDA, FINRA)	
domains	TEXT[]	Which domains to monitor for this source
is_active	BOOLEAN	

Field	Type	Notes
poll_interval_seconds	INTEGER, default 300	Matches the 5-minute polling requirement (G1)
last_polled_at	TIMESTAMPTZ, nullable	For the ETag-based “is this new” check
last_etag	TEXT, nullable	
updated_at	TIMESTAMPTZ	

In plain English: one row per regulator/domain combination being watched — lets you turn FINRA on/off, or restrict to just financial-domain filings, without redeploying code.

3.8 eval_runs

Standalone — periodic snapshots of system-wide quality metrics, not tied to any one filing.

Field	Type	Notes
id	UUID, PK	
run_type	ENUM(pre_deploy_regression, scheduled, manual)	
prompt_version	TEXT	Ties this run to a specific LangSmith pr
git_commit_sha	TEXT	Which code version was evaluated
ragas_faithfulness	FLOAT	Target >0.87 (PRD §10)
ragas_context_recall	FLOAT	Target >0.80
rouge_l	FLOAT	Target >0.45
alert_precision	FLOAT	Target <5% FP
alert_recall	FLOAT	Target <3% FN
hallucination_rate	FLOAT	Target <2%
extraction_f1	FLOAT	Target >0.82
p99_latency_ms	INTEGER	Target <500ms
avg_cost_per_filing_usd	NUMERIC(10,4)	Target <\$0.03
passed	BOOLEAN	Whether this run cleared all gating thro
created_at	TIMESTAMPTZ	

In plain English: every time the eval suite runs (in CI on a prompt change, or on a schedule), one row gets written here. This table is what GET /v1/metrics reads from, and it’s the log that lets you graph “did faithfulness drop after last week’s prompt change” over time.

3.9 api_keys

Authenticates API consumers (added beyond the PRD — see §1.10).

Field	Type	Notes
id	UUID, PK	
key_hash	TEXT	Hashed API key (never store plaintext)
owner_label	TEXT	Human-readable name (e.g., “Acme Corp Engineering”)
is_active	BOOLEAN	
rate_limit_per_minute	INTEGER, default 60	Per-key override for the rate limiter
created_at	TIMESTAMPTZ	
last_used_at	TIMESTAMPTZ, nullable	

In plain English: every request to a protected endpoint presents a key; this table is how the API knows who's calling and how hard to rate-limit them.

Relationships at a glance

- filings 1 → many filing_chunks
 - filings 1 → 1 extractions
 - filings 1 → 1 briefs
 - filings 1 → many deliveries
 - webhooks 1 → many deliveries (only for channel = webhook)
 - api_keys 1 → many webhooks
 - source_configs, eval_runs — standalone, no FK into filings
-

4. Environment Variables & Configuration Notes

4.1 Full .env reference

```
# — App —————
ENV=development                # development | staging | production
APP_SECRET_KEY=                # session/signing secret, generate with `openssl rand -hex 32`
LOG_LEVEL=INFO

# — Database (Supabase Postgres + pgvector) —————
DATABASE_URL=postgresql+asyncpg://user:pass@host:5432/regradar
DATABASE_POOL_SIZE=10
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=    # server-side only, never expose to any client

# — Redis (Celery broker + result backend) —————
REDIS_URL=redis://localhost:6379/0

# — Object storage (raw PDFs) —————
S3_BUCKET_NAME=regradar-filings-raw
S3_REGION=us-east-1
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=

# — LLM providers —————
OPENAI_API_KEY=                # GPT-4o (Analysis Agent) + text-embedding-3-small
HUGGINGFACE_API_TOKEN=        # bart-large-mnli, bart-large-cnn, Granite-13B via HF Inference API

# — Model routing tier thresholds —————
TIER_HIGH_MODEL=gpt-4o
TIER_LOW_MODEL=granite-13b
TIER_ROUTING_RISK_THRESHOLD=high # risk_level >= this uses TIER_HIGH_MODEL

# — Ingestion sources —————
SEC_EDGAR_USER_AGENT=          # EDGAR requires a descriptive User-Agent header on every request
SEC_EDGAR_RATE_LIMIT_PER_SEC=10
FDA_RSS_FEED_URL=
FINRA_FEED_URL=
INGESTION_POLL_INTERVAL_SECONDS=300

# — Prefect —————
PREFECT_API_URL=
```

```

PREFECT_API_KEY=

# — Delivery —————
SLACK_WEBHOOK_URL=                # default/fallback channel
SLACK_BOT_TOKEN=                  # if routing to multiple channels dynamically
SENDGRID_API_KEY=
SENDGRID_FROM_EMAIL=alerts@regradar.io
WEBHOOK_HMAC_ALGORITHM=sha256

# — Eval & observability —————
LANGSMITH_API_KEY=
LANGSMITH_PROJECT=regradar
LANGFUSE_PUBLIC_KEY=
LANGFUSE_SECRET_KEY=
LANGFUSE_HOST=https://cloud.langfuse.com
DAILY_COST_ALERT_THRESHOLD_USD=50 # circuit breaker trigger (PRD risk R4)

# — API / auth —————
API_RATE_LIMIT_PER_MINUTE_DEFAULT=60
API_KEY_HASH_ALGORITHM=bcrypt

# — Deployment —————
FLY_APP_NAME_API=regradar-api
FLY_APP_NAME_WORKER=regradar-worker
PORT=8000

```

4.2 Configuration notes before you start building

Secrets never go in the repo. Commit `.env.example` (with blank values, as above) and add `.env` to `.gitignore` on day one. Fly.io secrets should be set via `fly secrets set`, not baked into the Docker image.

SEC EDGAR requires a descriptive User-Agent. EDGAR will reject or rate-limit requests without a proper User-Agent header identifying your application and a contact email — set `SEC_EDGAR_USER_AGENT` to something like `"RegRadar/1.0 (sp3453@njit.edu)"` before the ingestion agent makes its first call, or every request will fail silently in a way that's confusing to debug later.

pgvector must be enabled explicitly on Supabase. It's an optional Postgres extension — run `CREATE EXTENSION IF NOT EXISTS vector;` as your first migration, before any table with a `VECTOR` column.

Two observability tools is a deliberate choice, not redundancy you should remove. LangSmith is for prompt versioning and eval regression (dev-time workflow); Langfuse is for the always-on cost/latency dashboard and the \$50/day budget circuit breaker (runtime workflow). If you want to cut scope, Langfuse can absorb both jobs with more manual setup — but don't wire eval-gating logic to Langfuse and cost-tracking to LangSmith; keep the split as described.

Tiered routing threshold is a config value, not a hardcoded branch. `TIER_ROUTING_RISK_THRESHOLD` should be read from config (or eventually `source_configs`-style table) rather than hardcoded in the Analysis Agent, since ADR-04's cost/quality tradeoff is exactly the kind of thing you'll want to tune after seeing real eval numbers.

HMAC secrets are per-webhook, not global. Each row in webhooks gets its own `hmac_secret` generated at registration time (`POST /v1/webhooks`) — don't reuse one global secret across all registered consumers, or one leaked webhook config compromises every consumer's ability to spoof payloads.

Rate limit config is per-key with a global default. `API_RATE_LIMIT_PER_MINUTE_DEFAULT` is the fallback; `api_keys.rate_limit_per_minute` overrides it per consumer. This lets you give a trusted internal consumer a higher limit without a code change.

Local dev should run entirely on docker-compose. Postgres+pgvector, Redis, the API, and one Cel-

ery worker should all be startable with `docker compose up` and zero external API calls required to get the pipeline running end-to-end against fixture PDFs (`tests/fixtures/sample_filings/`) — reserve real `OPENAI_API_KEY/HUGGINGFACE_API_TOKEN` usage for the first real integration test, not every local run.

5. What This Document Deliberately Doesn't Cover

Per the PRD's stated non-goals, this architecture excludes: a UI/dashboard (NG3 — API-first for V1), multi-tenant billing/SSO (V2), international regulator feeds (NG4), and OCR for scanned PDFs (cut from V1 scope, though `filing_chunks.is_table` and the `tests/fixtures/sample_filings/` edge-case corpus lay groundwork for adding it later without a schema change).